

Actividad 10: File Path Traversal Sequences
Stripped With Superfluous URL- Decode
Nivel-Practitioner

López Gómez Samuel-177393

CNO IV – Seguridad Informática

26/09/26

Servando López Contreras

Indice

Contenido

Indice	2
2. Marco teórico y explicación técnica de la vulnerabilidad	3
3. Procedimiento paso a paso y evidencias	4
3.1 Preparación del entorno de intercepción con Burp Suite	4
3.2 Acceso al laboratorio y generación de la instancia	6
3.3 Envío de una petición a Repeater	8
3.4 Intentos de explotación en Burp Repeater	10
3.5 Confirmación de la resolución del laboratorio	13
4. Explicación técnica de los payloads	14
4.1 Payload fallido 1 — /etc/passwd	14
4.2 Payload fallido 2 — ../../../../etc/passwd	14
4.3 Payload fallido 3 —//....//....//....//etc/passwd	14
4.4 Payload fallido 4 — codificación URL simple	14
4.5 Payload exitoso — codificación URL doble	15
5. Mitigación recomendada	15

2. Marco teórico y explicación técnica de la vulnerabilidad

Como en los laboratorios anteriores de la serie Path Traversal, la aplicación carga las imágenes de sus productos mediante un endpoint del tipo:

```
GET /image?filename=45.jpg
```

La particularidad de este laboratorio está en el orden en que se aplican sus dos mecanismos de defensa. Según la propia descripción del laboratorio, la aplicación primero bloquea cualquier entrada que contenga secuencias de traversal ("../") y, únicamente después de esa validación, realiza un URL-decode del valor recibido antes de usarlo para construir la ruta del archivo. A esa segunda decodificación —que ocurre después de que la petición ya fue decodificada una vez de forma automática por la capa HTTP— se le llama "superflua", porque la aplicación no debería necesitar decodificar el valor una segunda vez, y es precisamente ese paso adicional el que abre la puerta al bypass.

Para entender el fallo hay que separar dos capas de decodificación distintas:

- Decodificación automática de la capa HTTP: cuando el servidor recibe una petición, el propio framework decodifica una vez los caracteres porcentuales (%XX) del query string antes de exponer el valor a la lógica de la aplicación. Esto ocurre siempre, de forma transparente, para cualquier parámetro.
- Decodificación explícita y superflua de la aplicación: después de la validación anti-traversal, el código de la aplicación llama nuevamente a una función de URL-decode sobre el valor de filename, bajo el supuesto (incorrecto) de que todavía podría contener caracteres codificados.

El filtro anti-traversal se ejecuta entre esas dos decodificaciones, es decir, examina el valor tal como llega después de la primera decodificación (la automática), pero antes de la segunda (la explícita/superflua). Esto significa que basta con enviar la secuencia de traversal codificada con un nivel adicional de codificación URL para que el filtro no la reconozca como tal (porque en ese punto todavía contiene caracteres %XX y no la cadena literal "../"), y que sobreviva para ser decodificada por segunda vez justo antes de usarse, revelando entonces la secuencia real de traversal.

Esto explica por qué, durante la explotación, codificar la carga útil una sola vez no fue suficiente: la primera decodificación automática de la capa HTTP ya elimina ese único nivel de codificación antes de que el filtro actúe, dejando expuesta la secuencia "../" en texto plano, la cual es detectada y bloqueada de inmediato. Fue necesario aplicar un segundo nivel de codificación URL (codificar el propio carácter "%" como "%25") para que, tras la primera decodificación automática, el valor todavía contuviera secuencias %XX (y no "../" literal), pasando así el filtro sin ser detectado; la decodificación superflua que realiza la aplicación inmediatamente después es la que

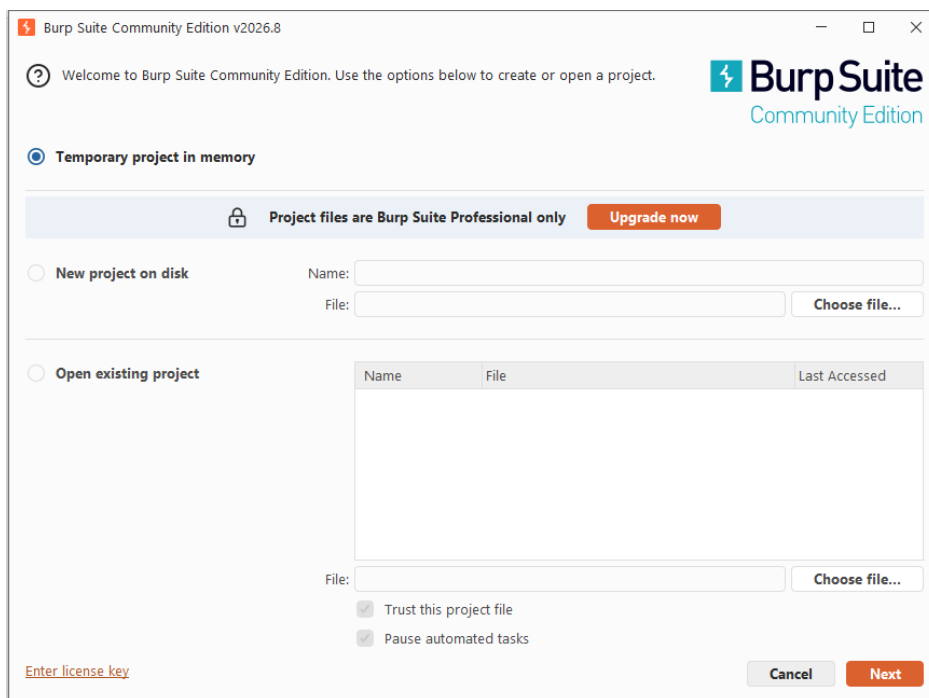
finalmente revela la secuencia de traversal real, ya sin ningún control posterior que la detenga.

Este comportamiento corresponde nuevamente a CWE-22 (Improper Limitation of a Pathname to a Restricted Directory), pero en una variante distinta a la de otros laboratorios de la serie: aquí la falla no está en la lógica de eliminación de patrones, sino en el orden de las operaciones de validación y transformación de la entrada. Es un recordatorio del principio de seguridad que establece que toda validación debe realizarse sobre la forma final y completamente normalizada de la entrada —después de cualquier decodificación—, nunca antes.

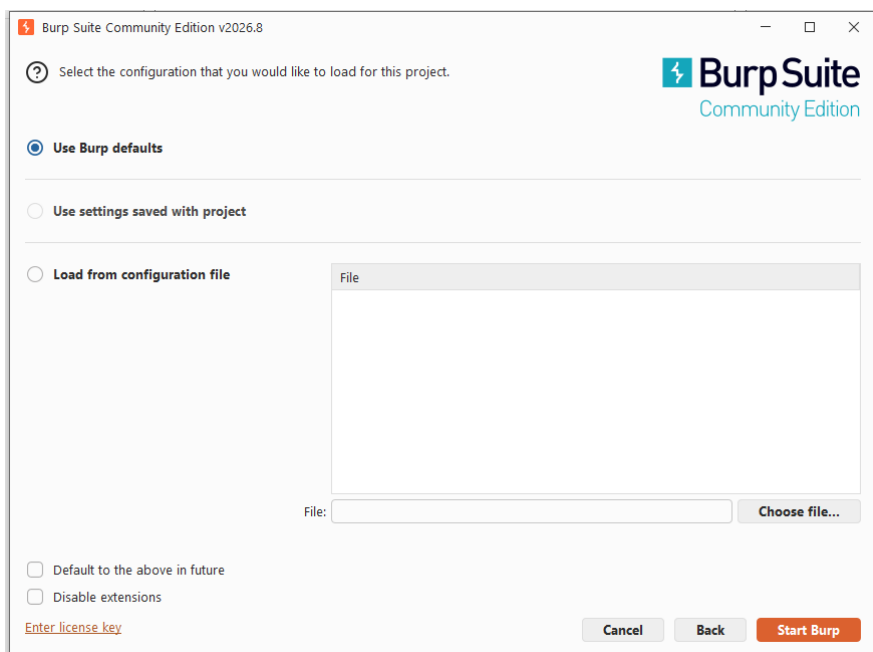
3. Procedimiento paso a paso y evidencias

3.1 Preparación del entorno de intercepción con Burp Suite

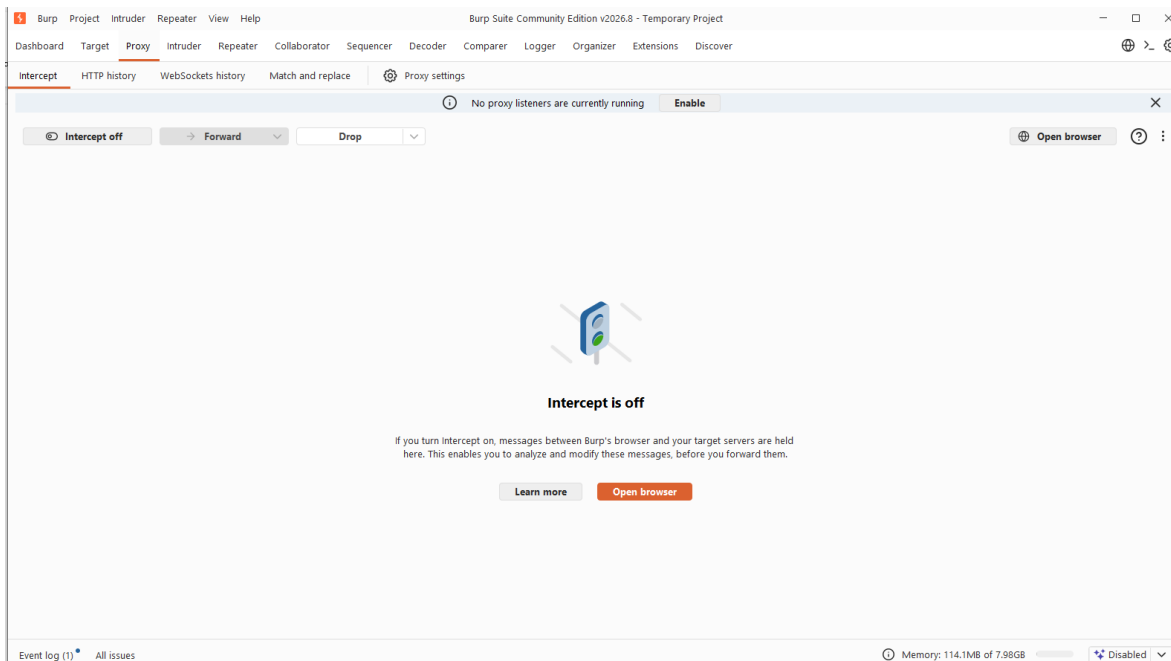
Paso 1. se abrió Burp Suite Community Edition y, en la ventana de bienvenida, se seleccionó "Temporary project in memory", adecuada para una sesión de práctica puntual que no requiere persistirse en disco.



Paso 2. en la siguiente ventana se eligió "Use Burp defaults" y se presionó "Start Burp", inicializando la herramienta con su configuración estándar

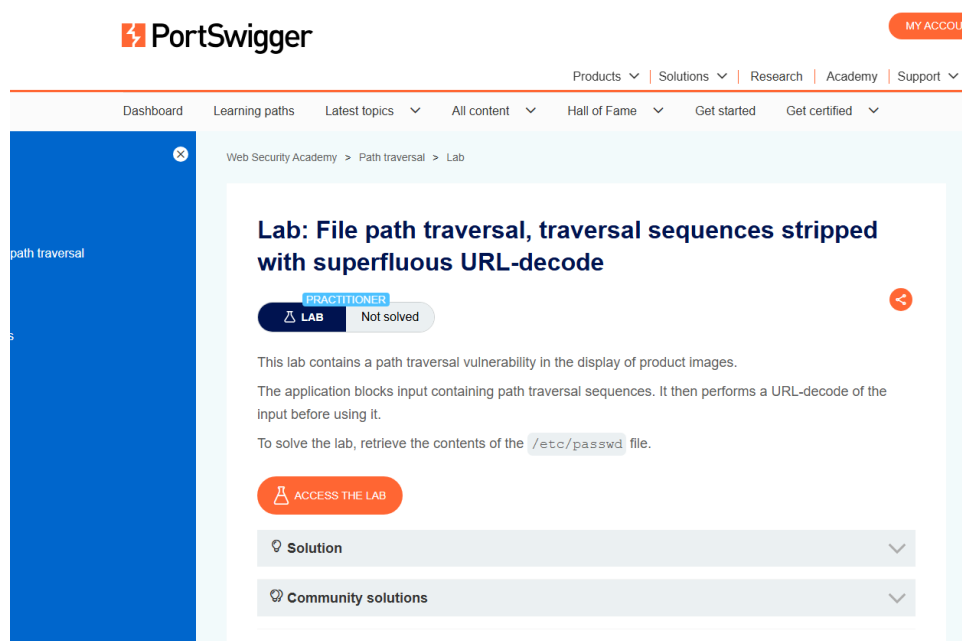


Paso 3. Burp abrió su panel principal en la pestaña Proxy > Intercept, con el interceptor apagado y sin listeners de proxy activos todavía, estado normal antes de comenzar a capturar tráfico.

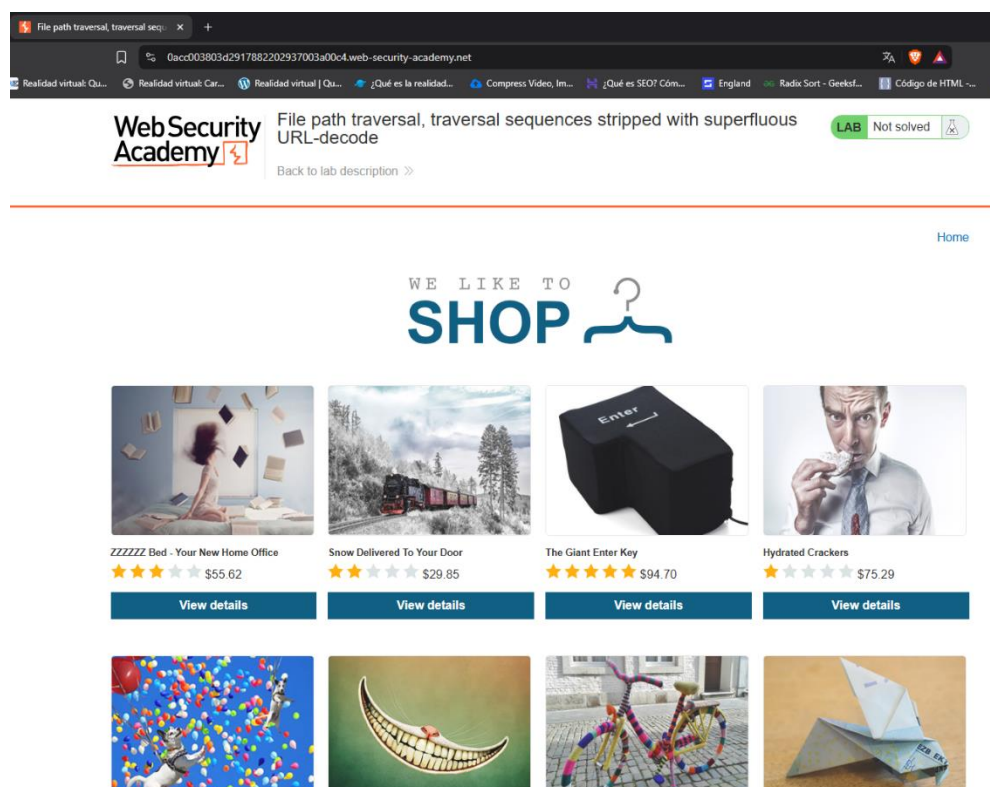


3.2 Acceso al laboratorio y generación de la instancia

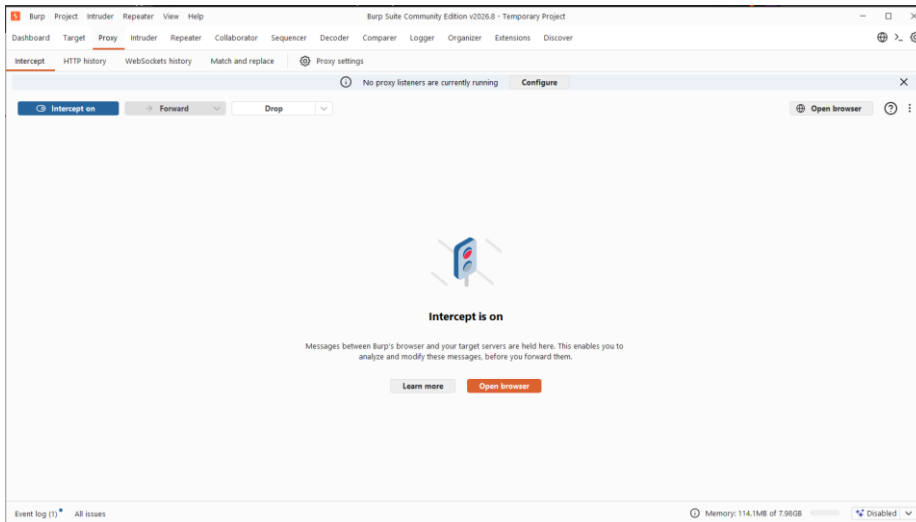
Paso 4. desde el navegador predeterminado se accedió a la página de descripción del laboratorio en PortSwigger Web Security Academy, donde se puede leer que la aplicación bloquea secuencias de traversal y luego realiza un URL-decode de la entrada, y que el objetivo es recuperar /etc/passwd. Se dio clic en "ACCESS THE LAB" para generar la instancia.



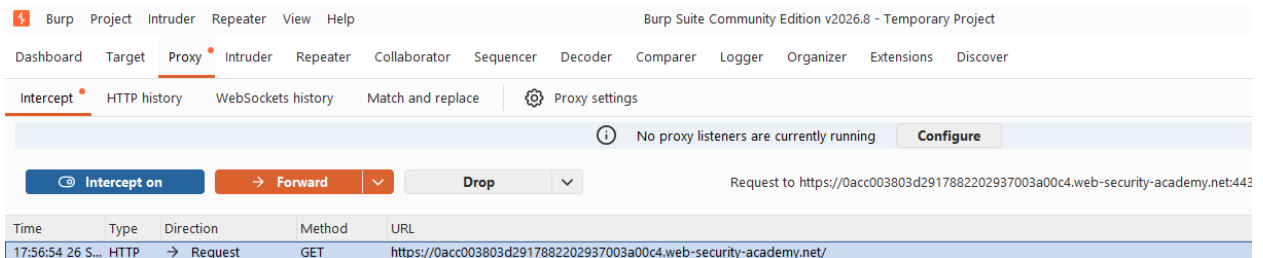
Paso 5. el navegador redirigió a la instancia única del laboratorio, con una URL aleatoria asociada a la sesión, mostrando la tienda "We Like To Shop" en estado "Not solved". De la barra de direcciones se copió dicha URL para reutilizarla en el navegador integrado de Burp.



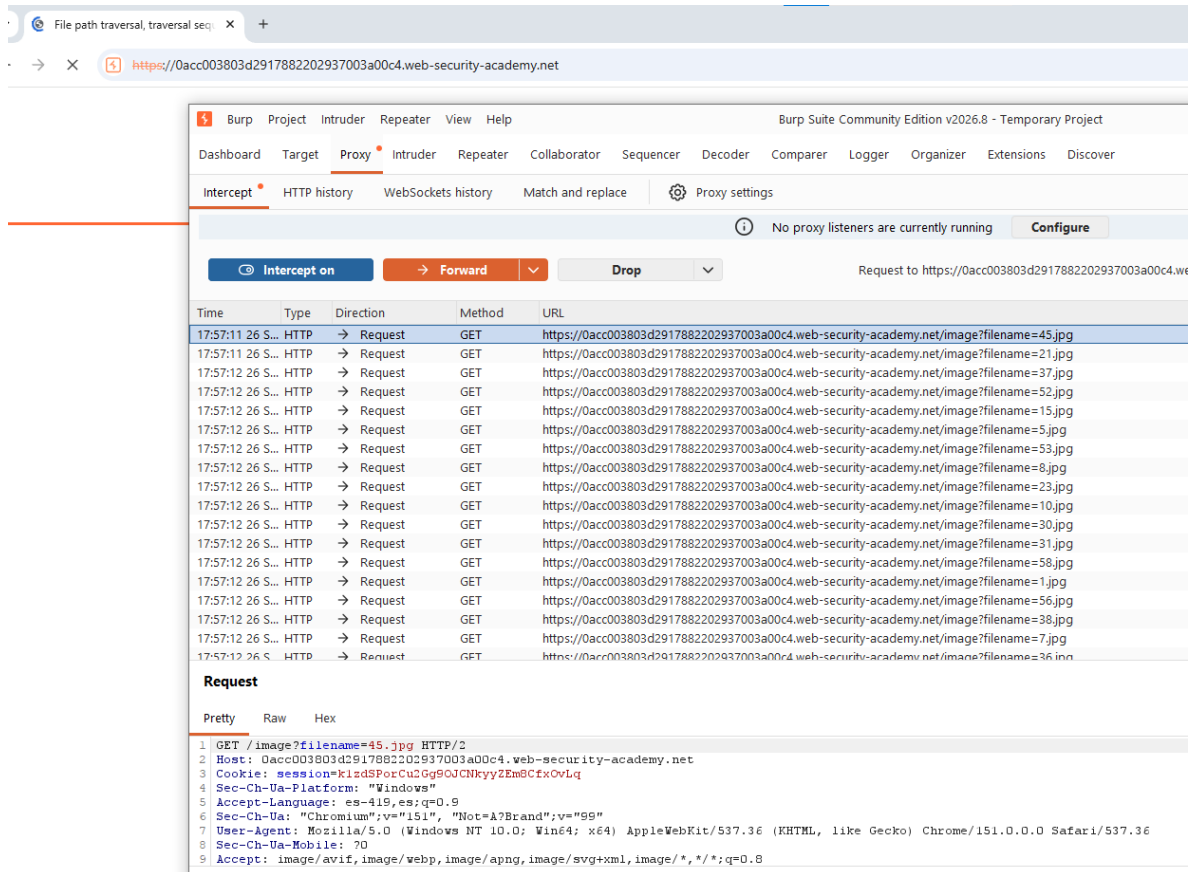
Paso 6. se regresó a Burp Suite y se activó "Intercept on" en la pestaña Proxy, de modo que toda petición quedara detenida hasta reenviarse manualmente. Después se utilizó "Open browser" para abrir la instancia de Chromium preconfigurada por Burp con el proxy y el certificado CA correctos.



Paso 7. en el navegador embebido de Burp se pegó la URL de la instancia obtenida en el paso 5. Al cargar la página, Burp interceptó de inmediato la primera petición: GET / dirigida al dominio de la instancia, correspondiente al documento HTML principal. Se presionó "Forward" para permitir que la carga continuara.



Paso 8. al seguir reenviando peticiones, Burp fue deteniendo, una a una, las numerosas solicitudes GET /image?filename=N.jpg generadas automáticamente para cargar las miniaturas del catálogo de productos (filename=45.jpg, 21.jpg, 37.jpg, 52.jpg, etc.), dejando en evidencia el patrón de referencia directa a archivos que utiliza la aplicación.

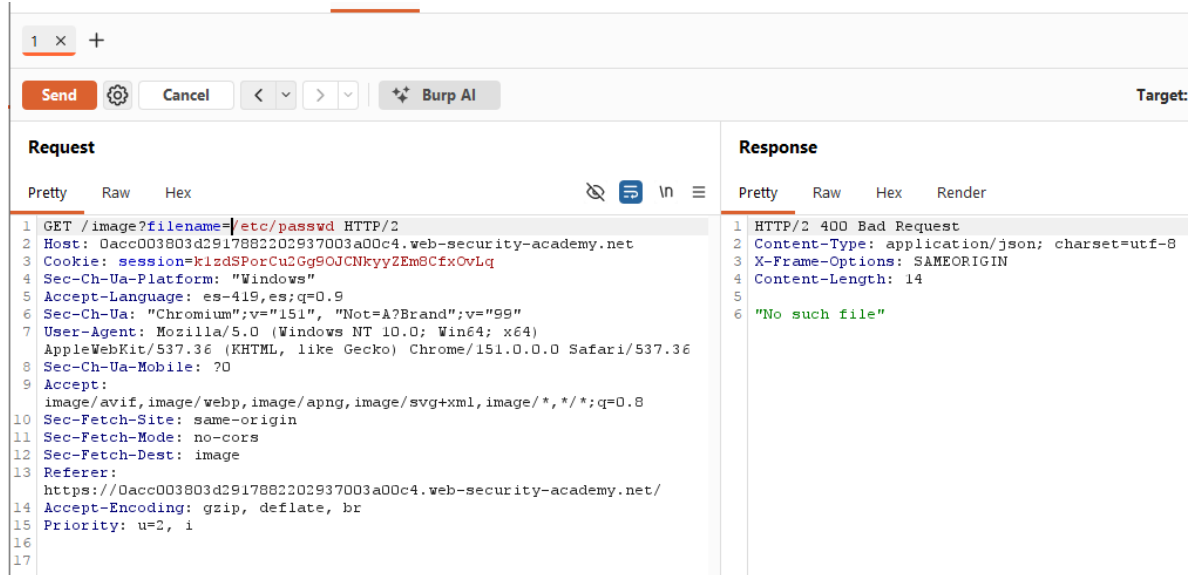


3.3 Envío de una petición a Repeater

Paso 9. sobre una de estas peticiones (filename=45.jpg) se dio clic derecho y se seleccionó "Send to Repeater" (Ctrl+R), enviando una copia exacta a la pestaña Repeater para su posterior manipulación manual.

3.4 Intentos de explotación en Burp Repeater

Paso 11. como primer intento se probó filename=/etc/passwd, una ruta absoluta simple sin codificación ni traversal relativo. El servidor respondió 400 Bad Request con el mensaje "No such file", descartando que el endpoint aceptara rutas absolutas sin más.



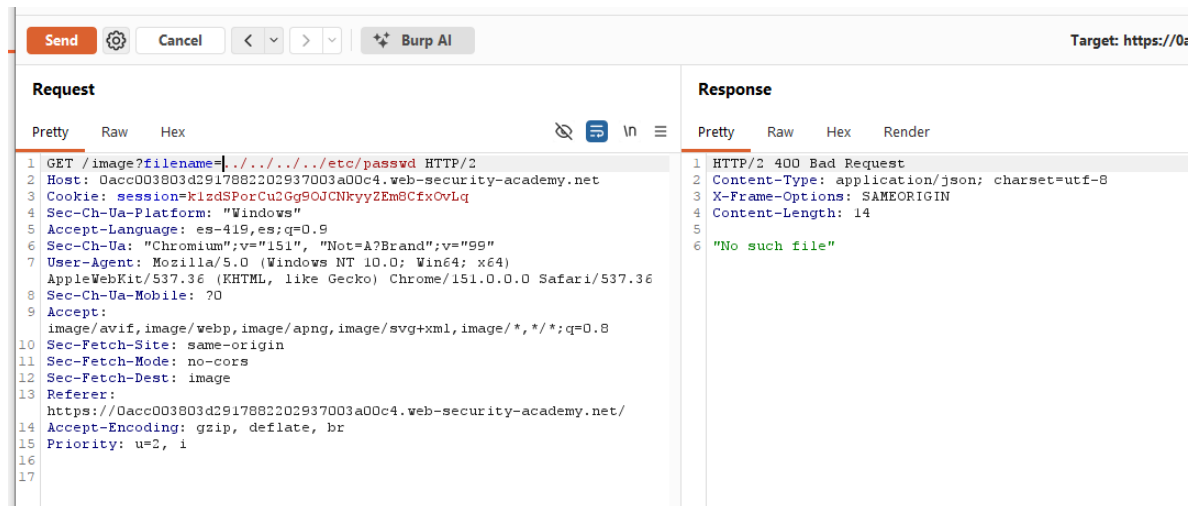
The screenshot shows the Burp Repeater interface. The 'Request' tab is selected, displaying the following details:

- Method: GET
- Path: /image?filename=/etc/passwd
- Host: 0acc003803d2917882202937003a00c4.web-security-academy.net
- Cookie: session=k1zdSPorCu2Gg9OJCNkyyZEm8CfxOvLq
- Sec-Ch-Ua-Platform: "Windows"
- Accept-Language: es-419,es;q=0.9
- Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
- Sec-Ch-Ua-Mobile: ?0
- Accept: image/avif, image/webp, image/apng, image/svg+xml, image/*;q=0.8
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: no-cors
- Sec-Fetch-Dest: image
- Referer: https://0acc003803d2917882202937003a00c4.web-security-academy.net/
- Accept-Encoding: gzip, deflate, br
- Priority: u=2, i

The 'Response' tab is also selected, showing the following details:

- Status: HTTP/2 400 Bad Request
- Content-Type: application/json; charset=utf-8
- X-Frame-Options: SAMEORIGIN
- Content-Length: 14
- Body: "No such file"

Paso 12. como segundo intento se utilizó el payload de traversal relativo clásico, filename=../../../../../etc/passwd. El servidor volvió a responder 400 Bad Request con "No such file", confirmando que la aplicación detecta y bloquea la secuencia "../" cuando aparece de forma literal en la entrada.



The screenshot shows the Burp Repeater interface. The 'Request' tab is selected, displaying the following details:

- Method: GET
- Path: /image?filename=../../../../../etc/passwd
- Host: 0acc003803d2917882202937003a00c4.web-security-academy.net
- Cookie: session=k1zdSPorCu2Gg9OJCNkyyZEm8CfxOvLq
- Sec-Ch-Ua-Platform: "Windows"
- Accept-Language: es-419,es;q=0.9
- Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
- Sec-Ch-Ua-Mobile: ?0
- Accept: image/avif, image/webp, image/apng, image/svg+xml, image/*;q=0.8
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: no-cors
- Sec-Fetch-Dest: image
- Referer: https://0acc003803d2917882202937003a00c4.web-security-academy.net/
- Accept-Encoding: gzip, deflate, br
- Priority: u=2, i

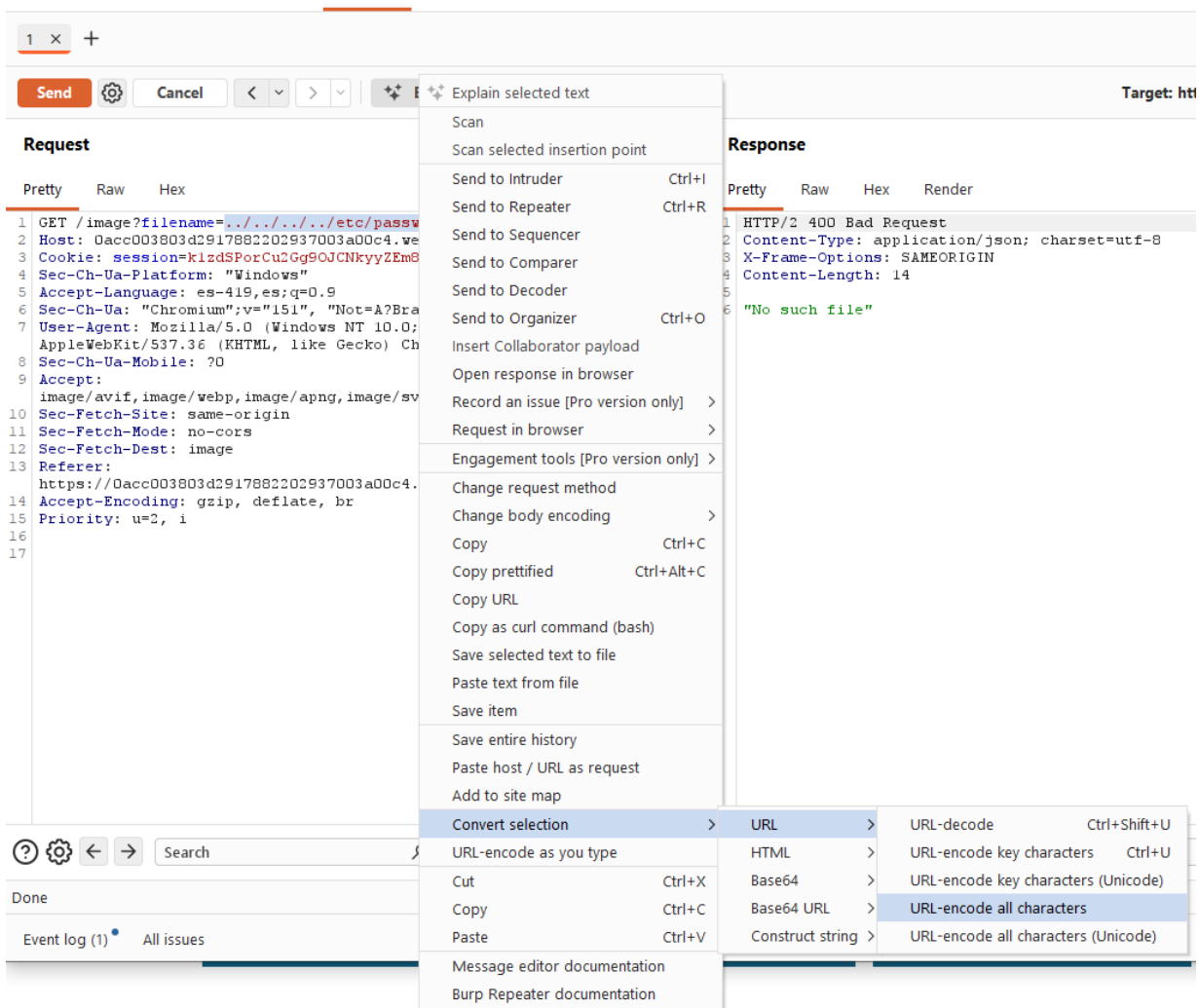
The 'Response' tab is also selected, showing the following details:

- Status: HTTP/2 400 Bad Request
- Content-Type: application/json; charset=utf-8
- X-Frame-Options: SAMEORIGIN
- Content-Length: 14
- Body: "No such file"

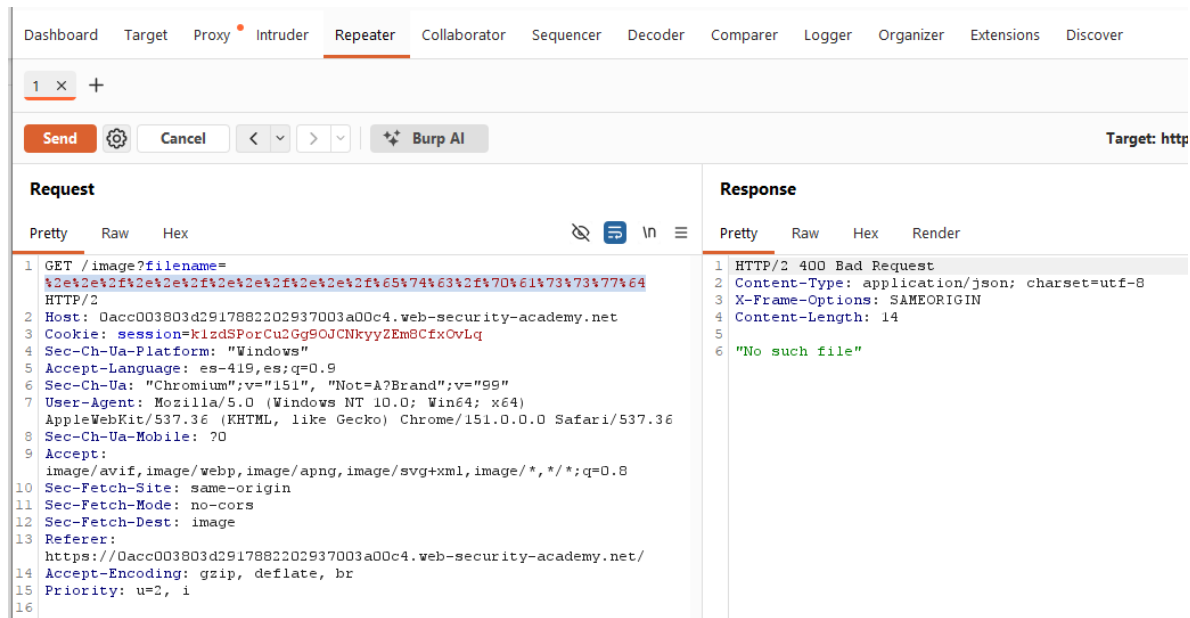
Paso 13. como tercer intento se probó la técnica de bypass por sanitización no recursiva vista en un laboratorio previo, filename=....//....//....//....//etc/passwd. El resultado fue nuevamente 400 Bad Request con "No such file": este laboratorio filtra las secuencias de traversal de una manera distinta (bloqueo directo del patrón, no una sola pasada de sustitución), por lo que esa técnica en particular no aplica aquí, lo cual en sí mismo es un hallazgo útil para descartar hipótesis.



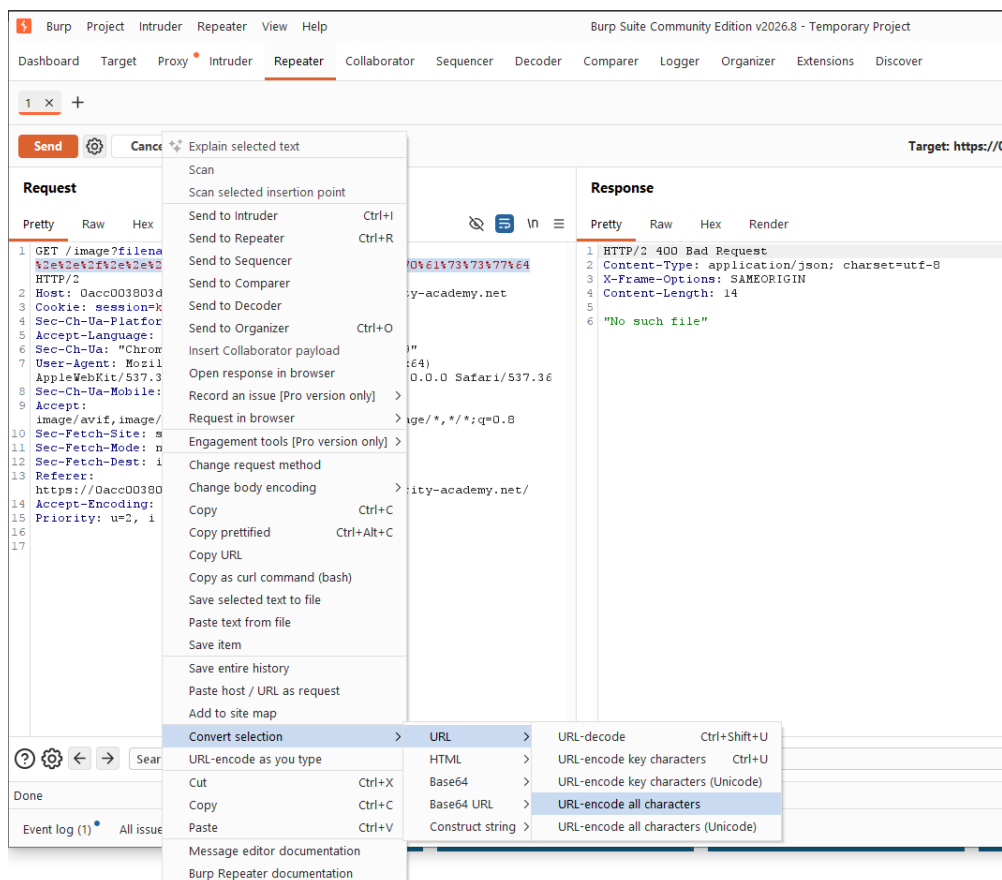
Paso 14. se seleccionó el texto ../../../../etc/passwd dentro del campo Request de Repeater y, mediante el menú contextual, se navegó a Convert selection › URL › "URL-encode all characters", lo que codificó cada carácter de la cadena a su representación porcentual (%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%65%74%63%2f%70%61%73%73%77%64).



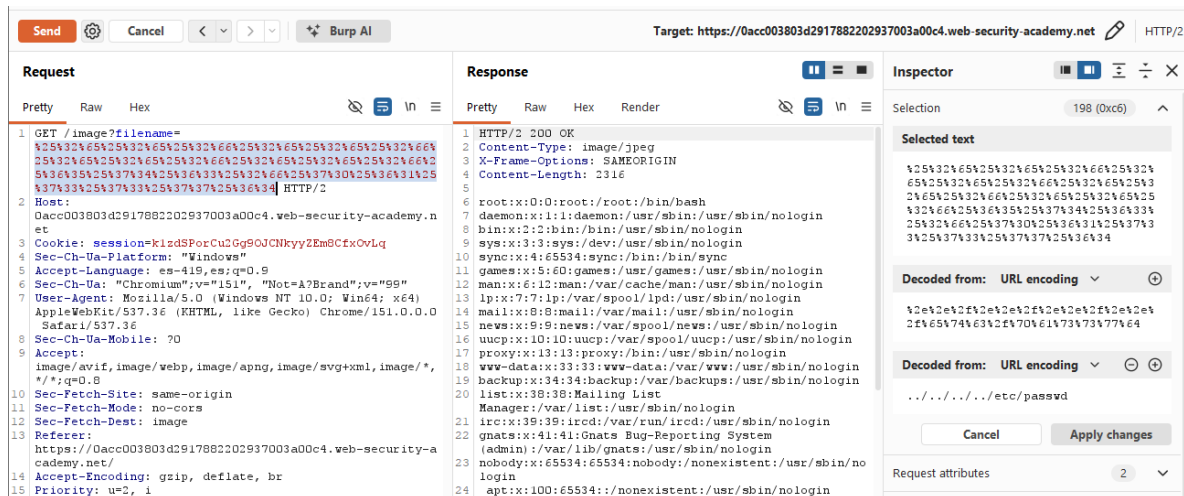
Paso 15. se envió la petición con la carga útil codificada una sola vez. El servidor respondió nuevamente 400 Bad Request con "No such file". Esto ocurre porque la capa HTTP decodifica automáticamente ese único nivel de codificación antes de que el filtro actúe, dejando expuesta la secuencia "../" en texto plano, la cual es detectada y bloqueada exactamente igual que en el paso 12.



Paso 16. se seleccionó el texto ya codificado (que en este punto contiene caracteres "%") y se le aplicó nuevamente Convert selection › URL › "URL-encode all characters", generando así una segunda capa de codificación (el carácter "%" de cada secuencia se codifica a su vez como "%25"), por ejemplo %2532%2e%2532%2e%2532%2566...

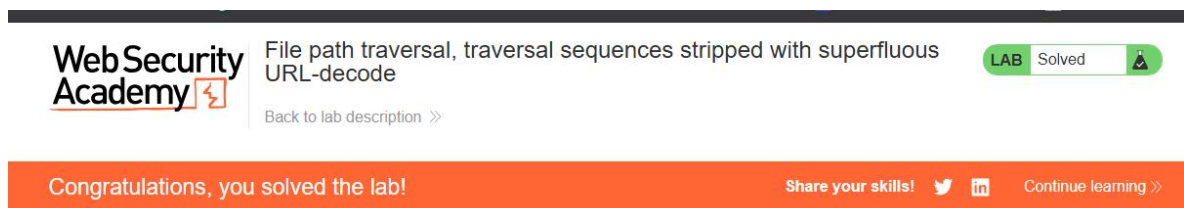


Paso 17. al enviar la petición con la carga útil doblemente codificada, el servidor respondió 200 OK con Content-Type: image/jpeg (nuevamente sin validar el contenido real de la respuesta), pero el cuerpo contenía el listado completo de cuentas del sistema operativo del servidor (root, daemon, bin, sys, www-data, backup, entre otras), es decir, el contenido real de /etc/passwd. El panel Inspector de Burp confirmó, mostrando la cadena "Decoded from: URL encoding" en dos niveles sucesivos, que el valor final decodificado era exactamente ../../../../etc/passwd.



3.5 Confirmación de la resolución del laboratorio

Paso 18. finalmente, se regresó a la pestaña del laboratorio en PortSwigger y se refrescó la página. La plataforma mostró el banner "Congratulations, you solved the lab!" y el estado cambió de "Not solved" a "Solved", acreditando la explotación exitosa de la vulnerabilidad.



4. Explicación técnica de los payloads

4.1 Payload fallido 1 — /etc/passwd

```
filename=/etc/passwd
```

Ruta absoluta sin codificación ni traversal relativo. La aplicación concatena este valor con su directorio base de imágenes, por lo que /etc/passwd no corresponde a ningún archivo existente dentro de esa carpeta; el servidor responde "No such file". Este intento sirvió para descartar que el endpoint resolviera rutas absolutas sin ninguna restricción.

4.2 Payload fallido 2 — ../../../../etc/passwd

```
filename=../../../../etc/passwd
```

Payload de traversal relativo estándar, en texto plano. La aplicación inspecciona la entrada, encuentra la secuencia literal "../" y bloquea la petición directamente, tal como anuncia el enunciado del laboratorio ("The application blocks input containing path traversal sequences").

4.3 Payload fallido 3 —//....//....//....//etc/passwd

```
filename=....//....//....//....//etc/passwd
```

Es la técnica de bypass por sanitización no recursiva (eliminar solo una vez la subcadena "../", dejando otra reconstruida) que sí funcionó en un laboratorio distinto de esta misma serie. Aquí no tuvo efecto porque el mecanismo de defensa de este laboratorio es diferente: no se trata de una sustitución de patrones que pueda reconstruirse, sino de un bloqueo directo de la entrada cuando contiene "../", seguido de una decodificación. Al no contener el patrón bloqueado en su forma literal ("....//", no es "../"), pero tampoco decodificarse a nada distinto de sí misma (no hay caracteres %XX que decodificar), la ruta resultante simplemente no corresponde a ningún archivo real.

4.4 Payload fallido 4 — codificación URL simple

```
filename=%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%65%74%63%2f%70%61%73%73%77%64
```

Es la secuencia ../../../../etc/passwd codificada una sola vez en URL. Falla porque la capa HTTP decodifica automáticamente los caracteres %XX del query string antes de que el valor llegue a la lógica de la aplicación; en el momento en que el filtro anti-traversal examina la entrada, esta ya se ve como ../../../../etc/passwd en texto plano, y por lo tanto es detectada y bloqueada exactamente igual que el payload del punto 4.2.

4.5 Payload exitoso — codificación URL doble

```
filename=%2532%2e%2532%2e%2532%2566%2532%2e%2532%2e%2532%2566%2532%2e%2532%2e%2532%2566%2532%2e%2532%2566%2532%2e%2532%2e%2532%2566%2537%2534%2536%2533%2532%2566%2537%2530%2536%2531%2537%2533%2537%2533%2537%2537%2536%2534
```

- Cada carácter de la carga útil original fue codificado dos veces: primero a su forma porcentual (".." → "%2e%2f") y, después, cada carácter "%" de esa forma fue codificado a su vez como "%25", produciendo secuencias como "%252e%252e%252f".
- Cuando la petición llega al servidor, la capa HTTP realiza su decodificación automática habitual: convierte "%25" en "%", eliminando así una sola capa de codificación. El resultado, en ese punto, es exactamente el payload de un solo nivel de codificación (%2e%2f...) — es decir, una cadena que todavía contiene caracteres porcentuales y no la secuencia literal "..".
- El filtro anti-traversal de la aplicación examina la entrada en este punto exacto (después de la única decodificación automática, antes de cualquier decodificación adicional del propio código de la aplicación) y, como no encuentra la subcadena literal "..", deja pasar la petición sin bloquearla.
- Inmediatamente después, la aplicación realiza su propia decodificación URL explícita y superflua sobre el valor ya "validado", convirtiendo %2e%2f%2e%2f... en la secuencia real ../../../etc/passwd, la cual se utiliza sin más controles para construir la ruta final del archivo a leer, logrando así el acceso arbitrario a /etc/passwd.

5. Mitigación recomendada

- Aplicar cualquier validación de seguridad (incluyendo la detección de secuencias de traversal) únicamente sobre la forma final, completamente decodificada y normalizada de la entrada, nunca antes de que se hayan aplicado todas las decodificaciones que el propio código vaya a ejecutar.
- Eliminar decodificaciones redundantes o superfluas del código: si el framework o servidor ya decodifica la entrada automáticamente, el código de la aplicación no debe volver a invocar una función de URL-decode sobre el mismo valor.
- Adoptar un enfoque de lista blanca (whitelist) para el nombre de archivo, validando contra un patrón estricto (por ejemplo, únicamente dígitos seguidos de ".jpg"), en lugar de intentar bloquear patrones específicos como "..", que pueden ser ofuscados mediante codificación.
- Canonicalizar la ruta resultante (equivalente a realpath()) después de todas las transformaciones y decodificaciones, y verificar que permanezca dentro del directorio base autorizado antes de acceder al archivo.
- Sustituir el nombre de archivo controlado por el usuario por un identificador indirecto (un ID numérico de producto), de modo que el servidor traduzca ese identificador a una ruta real sin exponer directamente el sistema de archivos a la entrada del cliente.
- Registrar y alertar sobre peticiones que contengan múltiples niveles de codificación URL en sus parámetros, ya que rara vez corresponden a un uso legítimo de la aplicación y son un indicador común de intentos de bypass.